Session 15 Teamwork — Give Every Real Write Route the Same Real Identity Check

Context

Work in your team (six teams of four students). Live code just built a real, working identity check end to end: `JwtAuthGuard` verifies a signed-in user's token, `@CurrentUser()` reads the identity it attaches to the request, and `GET /auth/me` proves the whole thing works. **Nothing in Issues, Projects, or Workspaces has been touched yet** — all three still use the old shared internal-tool key on their own write routes. This is this session's own main implementation block, not a short add-on: your team's job is applying the already-built guard, for the first time, to all three real resources, and proving it works — not inventing anything new.

Goal

By the end, every write route (`create`, `update`, `remove`) on **Issues**, **Projects**, and **Workspaces** requires a real, valid, signed-in user's token — the old shared internal-tool key alone is no longer enough on any of them — while `GET /projects/stats` is **completely untouched**, every read route stays open, and every other existing behavior across the whole app still works exactly as before.

What to build

Your team must:

- **Apply** the already-built `JwtAuthGuard` to the `create`, `update`, and `remove` routes on **Issues**, **Projects**, and **Workspaces** — nine routes total, three resources — replacing whatever currently guards them.
- **Update** each changed route's own Swagger documentation to describe what the new guard actually requires, and what it returns when that requirement isn't met — following the same pattern already shown live on `GET /auth/me`.
- **Leave `GET /projects/stats` completely untouched**, and **write down, in a code comment directly above it, why** — this is a required deliverable, not optional reading. Your explanation must correctly distinguish what `InternalToolGuard` answers from what `JwtAuthGuard` answers.
- **Preserve** every read route on all three resources exactly as it already behaves.
- **Be ready for the app to refuse to boot** the first time your team wires the guard into a module outside `AuthModule`. If that happens, this is a real Nest dependency-injection error caused by using `JwtAuthGuard` from a feature module for the first time — **read the error carefully before changing anything**; it names exactly what's missing and where. It is not a mistake in your own edit, and nothing in class has shown you this exact error yet — diagnosing it for real is part of the task.

Rules

Allowed: `JwtAuthGuard` and `@CurrentUser()` (already taught live and already proven on `GET /auth/me` — you do not need to write either one again, only use what already exists), `@ApiBearerAuth()`, everything already shown this session and in every prior session. No new packages, no new guard, no new decorator. If the app won't boot, the fix is a configuration change to existing, already-explained infrastructure — not a new mechanism.

Not yet: no `WorkspaceMember`, no roles, no ownership checks (assignee/reporter/Owner/Admin), no permission logic of any kind, no changes to `GET /projects/stats`'s own guard.

Test it (design your own full proof — this is a starting point, not the whole list)

BASH

```
# register and log in for a real token
curl -s -X POST http://localhost:3000/auth/register -H "Content-Type: application/json" \
  -d '{"name":"Test","email":"teamwork-check@example.com","password":"password123"}'
# ... then log in and extract accessToken, as already shown live

# for EACH of Issues, Projects, and Workspaces, prove all three cases on its own write
route:

# no token at all
curl -i -X POST http://localhost:3000/issues -H "Content-Type: application/json" -d '{}'

# the OLD key alone, no token
curl -i -X POST http://localhost:3000/issues -H "Content-Type: application/json" \
  -H "x-internal-tool-key: <the real key>" -d '{}'

# a real token, from a real login
curl -i -X POST http://localhost:3000/issues -H "Content-Type: application/json" \
  -H "Authorization: Bearer <a real token>" -d '{}'

# /projects/stats must behave the OPPOSITE way: the old key still works, a JWT alone does
not
curl -i http://localhost:3000/projects/stats
curl -i http://localhost:3000/projects/stats -H "Authorization: Bearer <a real token>"
curl -i http://localhost:3000/projects/stats -H "x-internal-tool-key: <the real key>"
```

Run the full set for all three resources, plus the `/projects/stats` contrast. Rejections must match the identical status/shape already shown live for every rejection reason — your team doesn't need to invent a new error response, only reuse the guard that already produces one.

Done when

- Every write route (`create` / `update` / `remove`) on Issues, Projects, and Workspaces rejects a request with no token, and separately rejects a request carrying only the old internal-tool key — both with the identical response already shown live for `GET /auth/me` 's own rejection cases.
- Every one of those same nine routes accepts a request carrying a real, valid token from a real login.
- `GET /projects/stats` still rejects a request with no key, still rejects a request carrying only a valid JWT, and still accepts the real internal-tool key — unchanged in every respect.
- A required, written comment directly above `GET /projects/stats` explains, correctly, why it deliberately keeps its original guard while every other write route in the app changed.
- Swagger's own documentation for every changed route accurately describes what it now requires, and the Swagger Authorize button can actually be used to call a protected route successfully from the browser.
- Every read route on all three resources, and every other previously working behavior in this app (registration, login, `/auth/me` , filtering), still behaves exactly as it did before your team's changes — proven, not assumed.
- Every team member can explain, in their own words, both (a) why `GET /projects/stats` was deliberately left alone, and (b) what the real boot error (if your team hit it) was actually telling you, and why the fix solves it.

Working time

About 60–70 minutes — the guard and the pattern already exist and are already proven once; most of the time is applying the pattern three times for real, diagnosing one genuinely real error along the way, writing the required audit, and proving all of it — not inventing anything new.